The Stack

Lecture 04

Data Structures and Algorithms

Part 1: What is a Stack?

The Stack — An Everyday Idea

Think about these real-world examples:

- **A stack of plates** in a cafeteria — you take from the top, you add to the top
- **Browser "Back" button** — each page you visit is pushed on; pressing back pops the last one
- **Undo (Ctrl+Z)** in a text editor — your most recent action is undone first
- **A stack of books** on your desk — you can only easily access the one on top

All of these follow the same principle:

The last item added is the first item removed.

This is the **LIFO** principle — **Last In, First Out**.

Stack as an Abstract Data Type

Recall from Lecture 03: an **ADT** defines *what* operations are available, not *how* they work internally.

Stack ADT Definition:

Component	Description
Data	A collection of elements with a designated "top"
Principle	LIFO — Last In, First Out

Stack as an Abstract Data Type

Operations:

Operation	Description
`push(x)`	Add element `x` to the top of the stack
`pop()`	Remove and return the top element
`peek()` / `top()`	View the top element without removing it
`isEmpty()`	Check if the stack has no elements
`isFull()`	Check if the stack is full (array-based only)
`size()`	Return the number of elements

Stack Operations: Visual Walkthrough

```
Start: Empty stack          push(10)          push(20)          push(30)
                           +------+          +------+          +------+
                           |  10  |          |  20  |  <-top   |  30  |  <-top
                           +------+          +------+          +------+
                             top             |  10  |          |  20  |
                                             +------+          +------+
                                                               |  10  |
                                                               +------+
```

```
pop() returns 30          peek() returns 20       pop() returns 20
+------+                  +------+                +------+
|  20  |  <-top           |  20  |  <-top         |  10  |  <-top
+------+                  +------+                +------+
|  10  |                  |  10  |
+------+                  +------+
```

Key observation: Only the **top** element is directly accessible at any time.

Overflow and Underflow

Two error conditions to watch out for:

Stack Overflow:

- Attempting to `push()` when the stack is already full
- Can only happen in array-based implementations (fixed size)
- The term "stack overflow" is also why the famous Q&A website got its name!

Stack Underflow:

- Attempting to `pop()` or `peek()` when the stack is empty
- Can happen in any implementation
- Must always check `isEmpty()` before popping

Overflow and Underflow

```
OVERFLOW:                                UNDERFLOW:
Stack is full, can't push!               Stack is empty, can't pop!

+------+                               +------+
|  40  | <-top                         | EMPTY |   pop() --> ERROR!
+------+                               +------+
|  30  |   push(50) --> ERROR!
+------+
|  20  |
+------+
|  10  |
+------+
```

Part 2: Stack Implementation Using Arrays

Array-Based Stack: Design

Idea: Use an array of fixed size `MAX` and a variable `top` to track the topmost element.

```
Array:  [ _ ][ _ ][ _ ][ _ ][ _ ]     MAX = 5
Index:    0    1    2    3    4
top = -1  (stack is empty)
```

Key Design Decisions:

- `top = -1` means the stack is empty
- `top = MAX - 1` means the stack is full
- Push: increment `top` first, then place the element
- Pop: read the element, then decrement `top`

```c
#define MAX 100

struct Stack {
    int arr[MAX];
    int top;
};
```

Array Stack: push() Operation

Algorithm:

1. Check if stack is full (`top == MAX - 1`) — if yes, overflow error
2. Increment `top`
3. Place element at `arr[top]`

```
void push(struct Stack* s, int value) {
    if (s->top == MAX - 1) {
        printf("Stack Overflow!\n");
        return;
    }
    s->top++;
    s->arr[s->top] = value;
}
```

Time Complexity: O(1) — constant time, always.

Array Stack: push() Visualization

```
Initial:    top = -1
[ _ ][ _ ][ _ ][ _ ][ _ ]

push(10):   top = 0
[10 ][ _ ][ _ ][ _ ][ _ ]
  ^
  top

push(20):   top = 1
[10 ][20 ][ _ ][ _ ][ _ ]
       ^
       top

push(30):   top = 2
[10 ][20 ][30 ][ _ ][ _ ]
            ^
            top
```

Each push simply increments `top` and writes to that position. No shifting needed!

Array Stack: pop() Operation

Algorithm:

1. Check if stack is empty (`top == -1`) — if yes, underflow error
2. Read element at `arr[top]`
3. Decrement `top`

```
int pop(struct Stack* s) {
    if (s->top == -1) {
        printf("Stack Underflow!\n");
        return -1;
    }
    int value = s->arr[s->top];
    s->top--;
    return value;
}
```

Time Complexity: O(1) — no shifting, no searching.

Note: We don't need to "erase" the old value. Decrementing `top` is enough — the position is now considered empty.

Array Stack: pop() Visualization

```
Before:      top = 2
[10 ][20 ][30 ][ _ ][ _ ]
            ^
            top

pop() → returns 30, top = 1
[10 ][20 ][30 ][ _ ][ _ ]     ← 30 is still in memory
       ^                          but logically removed
       top

pop() → returns 20, top = 0
[10 ][20 ][30 ][ _ ][ _ ]
  ^
  top

pop() → returns 10, top = -1
[10 ][20 ][30 ][ _ ][ _ ]     ← stack is now empty
 top = -1                        (even though values remain)
```

Array Stack: peek() and isEmpty()

```c
int peek(struct Stack* s) {
    if (s->top == -1) {
        printf("Stack is empty!\n");
        return -1;
    }
    return s->arr[s->top];
}

int isEmpty(struct Stack* s) {
    return s->top == -1;
}

int isFull(struct Stack* s) {
    return s->top == MAX - 1;
}

int size(struct Stack* s) {
    return s->top + 1;
}
```

All operations: O(1) — the hallmark of a well-designed stack.

Array Stack: Complete Example

```c
int main() {
    struct Stack s;
    s.top = -1;  // Initialize empty stack

    push(&s, 10);      // Stack: [10]
    push(&s, 20);      // Stack: [10, 20]
    push(&s, 30);      // Stack: [10, 20, 30]

    printf("%d\n", peek(&s));   // Output: 30
    printf("%d\n", pop(&s));    // Output: 30, Stack: [10, 20]
    printf("%d\n", pop(&s));    // Output: 20, Stack: [10]
    printf("%d\n", isEmpty(&s)); // Output: 0 (not empty)
    printf("%d\n", pop(&s));    // Output: 10, Stack: []
    printf("%d\n", isEmpty(&s)); // Output: 1 (empty)
}
```

Part 3: Stack Implementation Using Linked List

Why Use a Linked List?

The array-based stack works well, but has one significant limitation: **fixed size**.

Problem with Arrays	Solution with Linked List
Must define `MAX` at compile time	Grows dynamically — no predefined limit
May waste memory if MAX is too large	Uses only as much memory as needed
Stack Overflow if MAX is too small	No overflow (limited only by system memory)

Trade-off: Each element now carries extra memory for a pointer (4 or 8 bytes per node).

Linked List Stack: Design

Key Idea: The **top of the stack** is the **head of the linked list**.

- `push()` = insert at the head
- `pop()` = delete from the head

Both operations on a linked list's head are **O(1)** — perfect for a stack!

```
struct Node {
    int data;
    struct Node* next;
};

struct Node* top = NULL;  // Empty stack
```

```
Stack:   top --> [30|*]--> [20|*]--> [10|NULL]

Equivalent to:
  [30]  <-- top
  [20]
  [10]
```

Linked List Stack: push() Operation

Algorithm:

1. Create a new node with the given value
2. Set the new node's `next` to current `top`
3. Update `top` to point to the new node

```
void push(struct Node** top, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = *top;
    *top = newNode;
}
```

Time Complexity: O(1) — no traversal needed.

Linked List Stack: push() Visualization

```
Initial: top = NULL  (empty stack)

push(10):
  top --> [10|NULL]

push(20):
  newNode --> [20| *]
                  |
  top ---------->[20| *]--> [10|NULL]

push(30):
  top --> [30| *]--> [20| *]--> [10|NULL]
```

Each push creates a new node and places it at the front. The previous top becomes the second element.

Linked List Stack: pop() Operation

Algorithm:

1. Check if stack is empty (`top == NULL`) — if yes, underflow error
2. Save the top node's data
3. Move `top` to `top->next`
4. Free the old top node

```
int pop(struct Node** top) {
    if (*top == NULL) {
        printf("Stack Underflow!\n");
        return -1;
    }
    struct Node* temp = *top;
    int value = temp->data;
    *top = (*top)->next;
    free(temp);
    return value;
}
```


Time Complexity: O(1)

Linked List Stack: pop() Visualization

```
Before:
  top --> [30| *]--> [20| *]--> [10|NULL]

pop() returns 30:
  temp --> [30| *]   (to be freed)
  top ------------> [20| *]--> [10|NULL]
  free(temp)

pop() returns 20:
  temp --> [20| *]   (to be freed)
  top ------------> [10|NULL]
  free(temp)

pop() returns 10:
  temp --> [10|NULL]  (to be freed)
  top = NULL          (stack is empty)
  free(temp)
```

Important: Always `free()` the removed node to avoid memory leaks!

Linked List Stack: peek() and isEmpty()

```c
int peek(struct Node* top) {
    if (top == NULL) {
        printf("Stack is empty!\n");
        return -1;
    }
    return top->data;
}

int isEmpty(struct Node* top) {
    return top == NULL;
}
```

No `isFull()` needed — the linked list stack can grow until system memory runs out.

Part 4: Comparing the Two Implementations

Array vs. Linked List Stack

Feature	Array Implementation	Linked List Implementation
Size	Fixed (`MAX`)	Dynamic (grows as needed)
Memory usage	May waste space	No wasted space, but pointer overhead
Overflow	Possible (`top == MAX-1`)	Only if system memory runs out
Speed	Slightly faster (cache-friendly)	Slightly slower (pointer chasing)
Code complexity	Simpler	Slightly more complex
Memory allocation	Static (compile time)	Dynamic (`malloc` / `free`)

When to Use Which?

Choose Array-Based Stack when:

- Maximum size is known in advance
- Performance is critical (cache locality matters)
- Simplicity of code is preferred
- Memory overhead per element should be minimal

Choose Linked List Stack when:

- Size is unpredictable or highly variable
- You cannot afford to pre-allocate memory
- Stack overflow must be avoided at all costs
- Frequent resizing would be needed with arrays

In practice: Array-based stacks are more common due to their simplicity and speed. Linked list stacks are used when flexibility is essential.

Part 5: Tracing Through Examples

Trace Exercise 1

What does the stack look like after these operations?

```
push(5)  →  push(10)  →  pop()  →  push(15)  →  push(20)  →  pop()
```

Step-by-step trace:

```
push(5):    [5]            top=0
push(10):   [5][10]        top=1
pop():      [5]            top=0,  returned 10
push(15):   [5][15]        top=1
push(20):   [5][15][20]    top=2
pop():      [5][15]        top=1,  returned 20
```

Final Stack: `[5, 15]` with `15` on top.

Trace Exercise 2

What is the output of the following code?

```
struct Stack s;
s.top = -1;

push(&s, 1);
push(&s, 2);
push(&s, 3);
printf("%d ", pop(&s));
printf("%d ", pop(&s));
push(&s, 4);
push(&s, 5);
printf("%d ", pop(&s));
printf("%d ", pop(&s));
printf("%d ", pop(&s));
```

Trace Exercise 2: Solution

```
push(1):            [1]                    top=0
push(2):            [1][2]                 top=1
push(3):            [1][2][3]              top=2
pop() → print 3:    [1][2]                top=1
pop() → print 2:    [1]                   top=0
push(4):            [1][4]                 top=1
push(5):            [1][4][5]              top=2
pop() → print 5:    [1][4]                top=1
pop() → print 4:    [1]                   top=0
pop() → print 1:    []                    top=-1
```

Output: `3 2 5 4 1`

Notice: elements come out in the **reverse order** of how they went in. This reversal property is fundamental to many stack applications!

Trace Exercise 3: Linked List Stack

Trace `push(A)`, `push(B)`, `push(C)`, `pop()`, `push(D)`:

```
push(A):
  top --> [A|NULL]

push(B):
  top --> [B|*]--> [A|NULL]

push(C):
  top --> [C|*]--> [B|*]--> [A|NULL]

pop() returns C:
  top --> [B|*]--> [A|NULL]     (C node freed)

push(D):
  top --> [D|*]--> [B|*]--> [A|NULL]
```

Final Stack (top to bottom): D, B, A

Key Takeaways

What We Learned Today

1. Stack Concept

- LIFO (Last In, First Out) principle
- Only the top element is directly accessible
- Core operations: `push()`, `pop()`, `peek()`, `isEmpty()`
- Error conditions: overflow (push on full) and underflow (pop on empty)

2. Array Implementation

- Fixed-size array with a `top` index (initialized to -1)
- All operations O(1) — increment/decrement `top` and read/write
- Simple and cache-friendly, but limited by `MAX` size

3. Linked List Implementation

- Dynamic size using nodes with `data` and `next` pointer
- `top` pointer = head of the list; push/pop at head
- All operations O(1) — no traversal needed
- No overflow, but has pointer overhead per node

Complexity Summary

Operation	Array Stack	Linked List Stack
`push()`	O(1)	O(1)
`pop()`	O(1)	O(1)
`peek()`	O(1)	O(1)
`isEmpty()`	O(1)	O(1)
`isFull()`	O(1)	N/A
Space	O(n) fixed	O(n) dynamic

Both implementations deliver the same O(1) performance. The choice between them is about trade-offs: simplicity and speed (array) vs. flexibility and dynamic sizing (linked list).